IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

High-Level Design

for

Zero-Day Scanner for Agriculture Web Portals

Giridhar Pai

23BCCED015

BCA in Cyber Security, Ethical Hacking & Digital Foreignsic with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

15-04-2026

Under the guidance of

Mr. Vikyath

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction

1.1 Scope of the Document

The High-Level Design document for Zero-Day Scanner for Agriculture Web Portals (Tejas Raksha) provides a comprehensive architectural blueprint that details complete system architecture with component interactions, data flow patterns for synchronous and asynchronous operations, database schema design with relationships and constraints, API specifications with request/response contracts, security mechanisms and authentication flows, performance optimisation strategies, scalability and deployment patterns, and integration points with external systems.

1.2 Intended Audience

Audience	Purpose
Solution Architects	Understand design patterns, technology choices, and system evolution
Technical Leads	Plan implementation phases, resource allocation, risk assessment
Senior Developers	Understand component interactions and integration points
DevOps Engineers	Infrastructure requirements, deployment architecture, monitoring
Security Auditors	Review security controls, authentication flows, data protection
Product Managers	Understand technical capabilities and feature feasibility
QA Teams	Plan test scenarios, integration testing, performance validation

1.3 System Overview

Zero-Day Scanner for Agriculture Web Portals Tejas Raksha is a distributed, microservices-based web application security scanner designed to automatically discover and report vulnerabilities in web applications, aligned with OWASP guidelines.

Attribute	Details
Project Name	Tejas Raksha (तेजस रक्षा)
Meaning	"Tejas" = Brilliance/Radiance, "Raksha" = Protection
Type	Web Application Security Scanner
Status	100% Complete & Operational
Deployment	Containerised (Docker + Docker Compose)

Core Capabilities:

- **Automated Security Testing** – Continuous scanning, scheduled runs (daily/weekly/monthly), real-time detection.
- **Intelligent Detection** – 15+ vulnerability categories including XSS, SQLi, CSRF, SSRF, XXE, command injection, authentication bypass, insecure deserialisation, directory listing, sensitive file exposure, open redirect, missing security headers.
- **False Positive Reduction** – Multi-stage verification reduces false positives by 70-85%; confidence scoring (0-100) ensures only findings with score ≥70 are reported.
- **Enterprise Features** – Multi-user support (API keys), Slack/Jira/webhook integrations, PDF reports with branding, historical trend analysis, CVE database enrichment, email notifications

Technology Stack:

Tier	Technologies
Frontend	React 18, Vite, TailwindCSS, Phosphor Icons, Axios, React Router v6, WebSocket API
Backend	FastAPI, Python 3.11, Uvicorn, Pydantic v2, SQLAlchemy 2.0, Alembic
Task Processing	Celery 5.3, Redis 7 (broker & result backend)
Database	PostgreSQL 15
Security Scanning	BeautifulSoup4, Requests, aiohttp
Reporting & Integration	ReportLab, SMTP, Slack SDK, Jira API
DevOps	Docker 24.0, Docker Compose, NGINX, Let's Encrypt

2. System Design

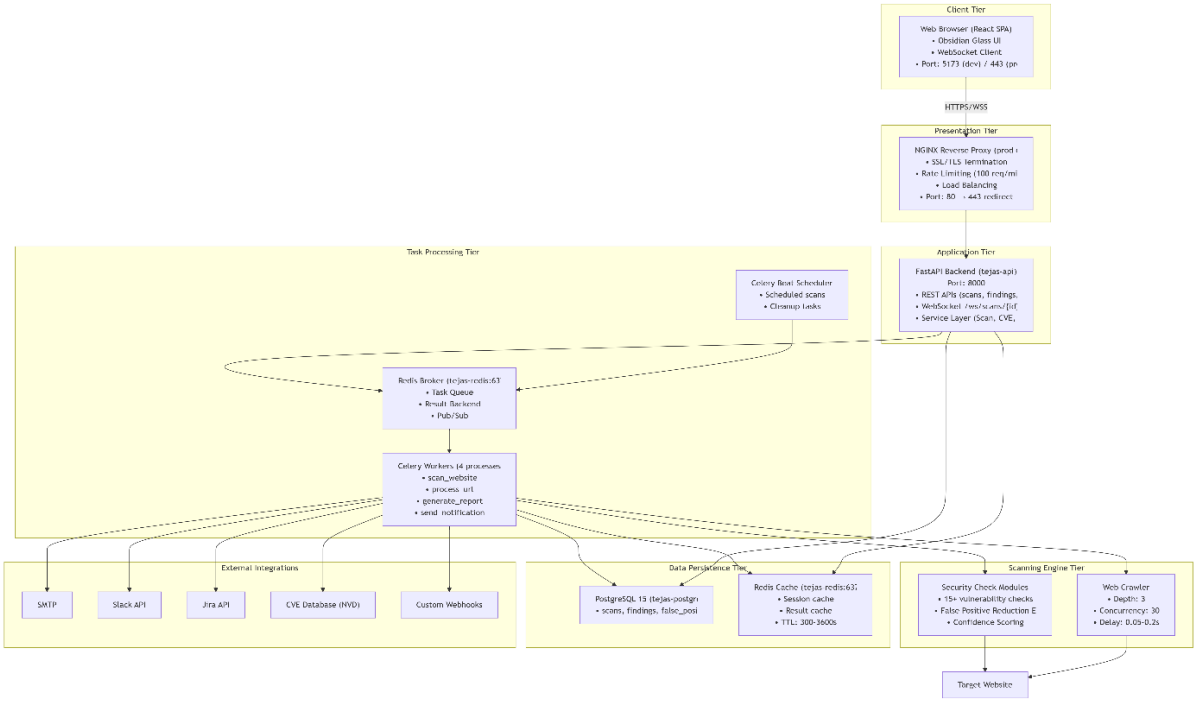
2.1 Application Design

The system follows a microservices architecture with clear separation between presentation, application, task processing, scanning engine, data persistence, and external integrations.

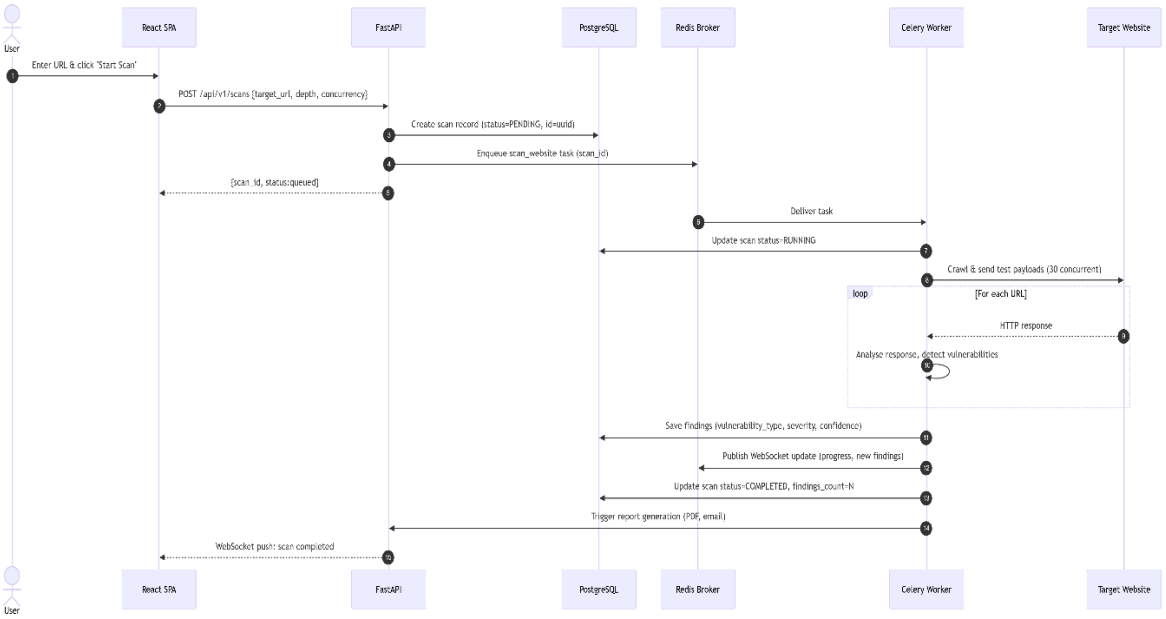
Container Names & Network:

- Containers: tejas-frontend, tejas-api, tejas-worker, tejas-beat, tejas-postgres, tejas-redis

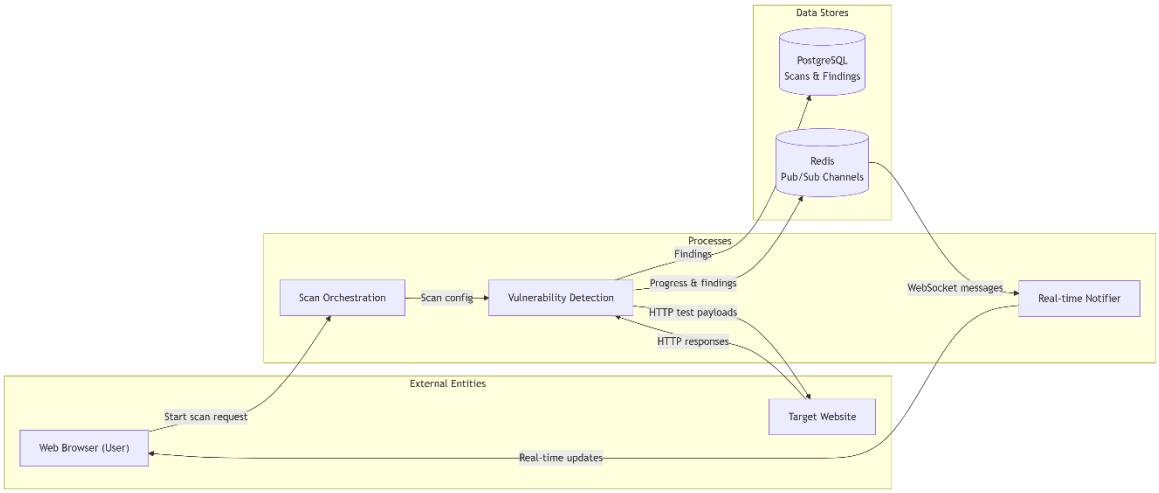
- Network: tejas-network (bridge) – internal communication only; frontend and API exposed to host.



2.2 Process Flow (Scan Lifecycle) – Sequence Diagram



2.3 Information Flow (Real-time Updates) – Data Flow Diagram



2.4 Components Design (Detailed)

Component	Technology	Key Responsibilities
Frontend SPA	React 18 + Vite	Render UI, manage state, WebSocket client, report viewer
NGINX Proxy	NGINX 1.24	SSL termination, rate limiting (100 req/min), static asset serving
FastAPI Backend	FastAPI + Uvicorn	REST endpoints, WebSocket server, request validation, service orchestration
Celery Worker	Celery 5.3 + Python	Execute `scan_website`, `process_url`, `generate_report`, `send_notification`
Celery Beat	Celery Beat	Schedule recurring scans, data cleanup, health checks
Redis Broker	Redis 7	Task queue, result backend, Pub/Sub channels
PostgreSQL	PostgreSQL 15	Persistent storage for scans, findings, false positives, schedules

Redis Cache	Redis 7	Session cache, result cache (TTL 300–3600s)
Scanning Engine	Custom Python modules	15+ security checks, web crawler, false positive reduction
PDF Generator	ReportLab	Generate branded PDF reports with findings and remediation

2.5 Key Design Considerations

1. **Asynchronous Processing –** Long-running scans are offloaded to Celery workers, keeping the API responsive.
2. **False Positive Reduction –** Multi-stage verification with baseline 404 detection, content signature matching, and confidence scoring (only findings with confidence ≥70 are reported).
3. **Real-time Updates –** WebSocket + Redis Pub/Sub provides live progress without client-side polling.
4. **Scalability –** Stateless API and workers allow horizontal scaling; connection pooling for PostgreSQL and Redis.
5. **Security –** API key authentication (bcrypt hashed), CORS restrictions, rate limiting, and security headers (HSTS, CSP).
6. **Extensibility –** Modular security checks (one Python file per vulnerability type); new checks can be added without changing the core engine.

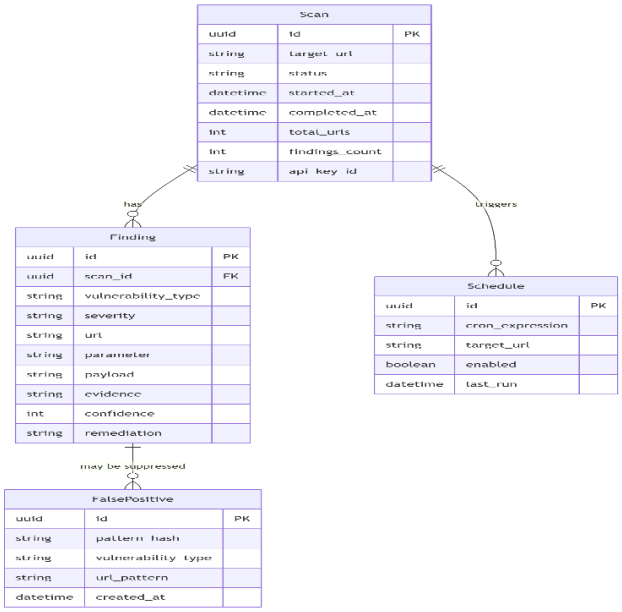
2.6 API Catalogue

Endpoint	Method	Description
`/api/v1/scans`	POST	Create a new scan
`/api/v1/scans`	GET	List all scans
`/api/v1/scans/{id}`	GET	Get scan details
`/api/v1/scans/{id}/results`	GET	Get findings for a scan
`/api/v1/scans/{id}`	DELETE	Delete a scan

`/api/v1/false-positives`	GET	List false positive suppressions
`/api/v1/false-positives`	POST	Mark a finding as false positive
`/api/v1/schedules`	GET	List scheduled scans
`/api/v1/schedules`	POST	Create a schedule
`/api/v1/trends`	GET	Get historical vulnerability trends
`/api/v1/comparison`	GET	Compare two scans
`/api/v1/cve/search`	GET	Search CVE database (NVD)
`/api/v1/integrations`	GET	List configured integrations
`/reports/pdf/{id}`	GET	Generate PDF report
`/ws/scans/{scan_id}`	WebSocket	Real-time scan progress and findings

3. Data Design

3.1 Entity-Relationship (ER) Diagram



3.2 Data Model (Schema & Encryption)

Scan Table

Column	Type	Description
`id`	UUID	Primary key
`target_url`	TEXT	Target website URL
`status`	VARCHAR(20)	`pending` / `running` / `completed` / `failed`
`started_at`	TIMESTAMP	Scan start time
`completed_at`	TIMESTAMP	Scan completion time
`total_urls`	INTEGER	Number of URLs crawled
`findings_count`	INTEGER	Number of findings detected
`api_key_id`	VARCHAR(100)	API key used (audit trail)

Finding Table

Column	Type	Description
`id`	UUID	Primary key
`scan_id`	UUID	Foreign key to `scans`
`vulnerability_type`	VARCHAR(50)	e.g., `xss`, `sql_injection`
`severity`	VARCHAR(20)	`critical` / `high` / `medium` / `low` / `info`
`url`	TEXT	Vulnerable URL
`parameter`	TEXT	Parameter name (if applicable)
`payload`	TEXT	Payload that triggered the finding
`evidence`	TEXT	Snippet of response evidence
`confidence`	INTEGER	0–100 (only ≥70 reported)
`remediation`	TEXT	Fix recommendation

3.3 Data Access Mechanism

- No database used. All data is held in memory as Python objects.
- Reports are written to the filesystem via standard file I/O.
- Configuration is read from YAML (optional) or CLI arguments.
- Logs are written to console and optionally to a log file.

Encryption: No sensitive user data is stored. API keys are hashed using bcrypt (cost factor 12). No encryption at rest is required for findings (they are security test results, not personal data).

3.4 Data Retention & Archival Policies

Data Entity	Retention Period	Enforcement
Scan records	90 days	Automated daily cleanup task
Findings	90 days (cascade delete with scan)	Automated
False positives	Indefinite	Manually managed
Schedules	Indefinite	User-managed
PDF reports	User-managed	Stored on filesystem; no auto-delete

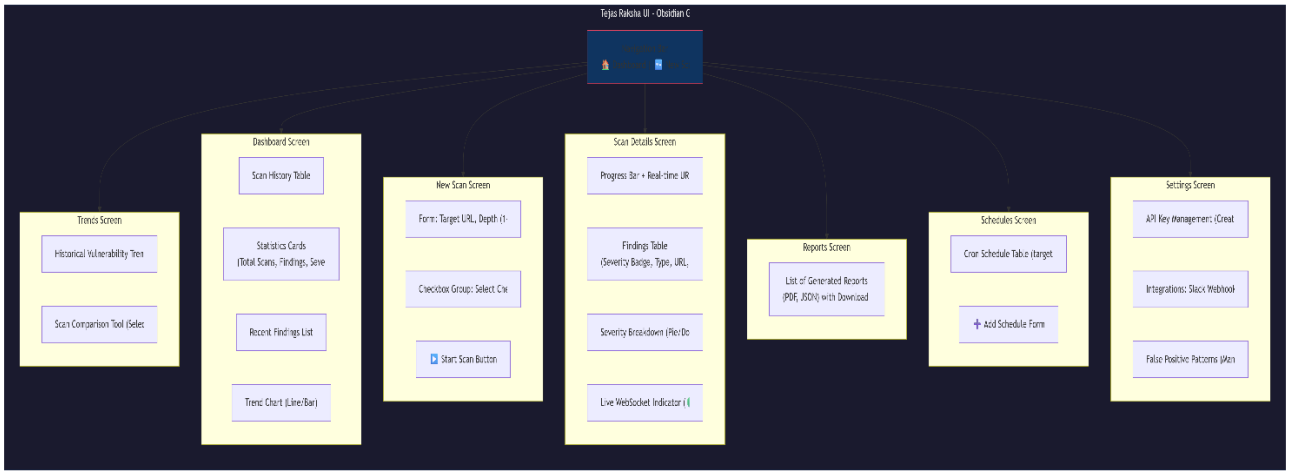
3.5 Data Migration & Seeding

- **Migrations:** alembic upgrade head
- **Seeding script:** scripts/seed_db.py creates:
    - Default API keys (development only)

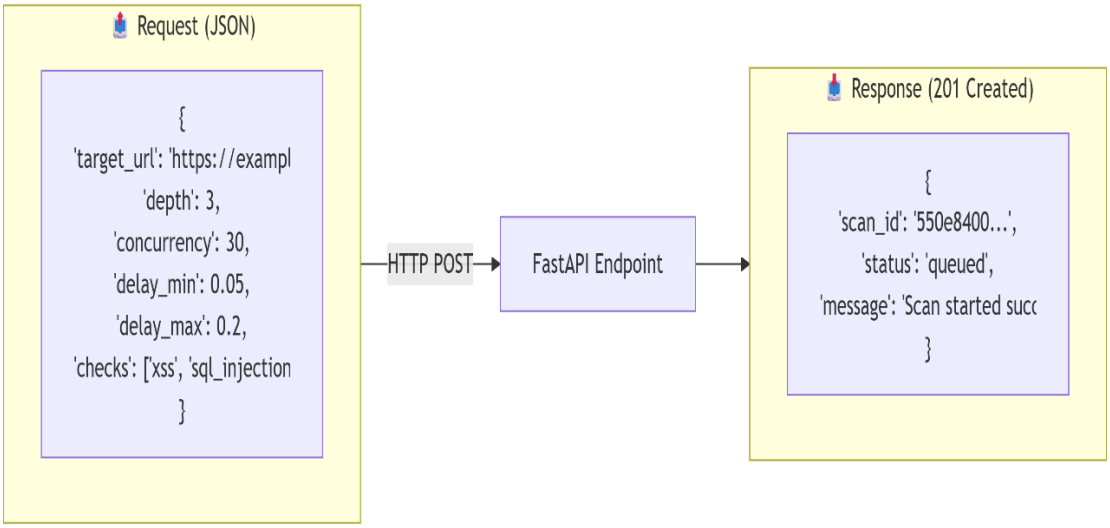
- Example schedules (daily scan of test target)
- Built-in false positive patterns (17+ patterns)

4. Interfaces

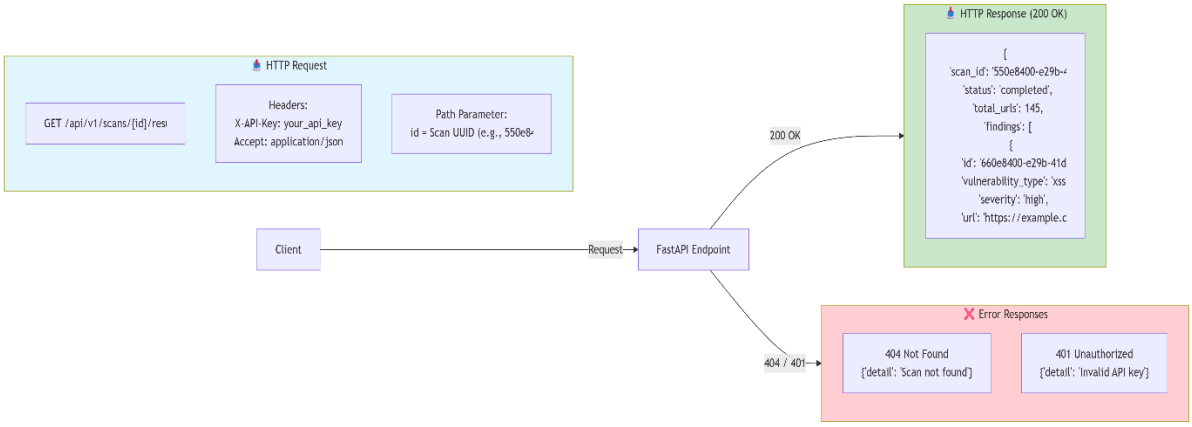
4.1 User Interface (UI) Layout



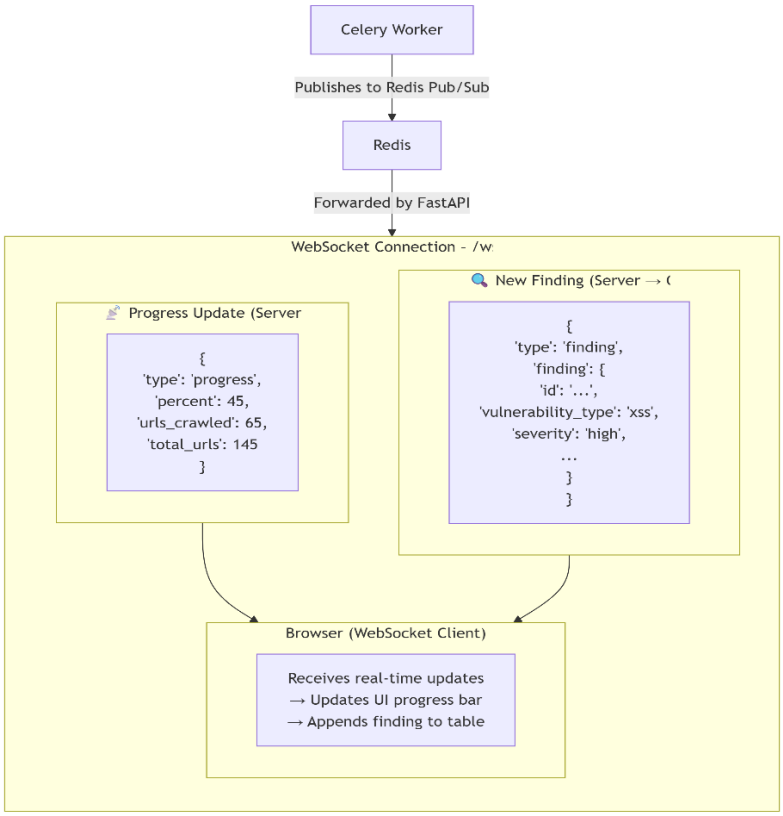
4.2 API Contracts (Request / Response Structure)



GET /api/v1/scans/{id}/results – Get findings



WebSocket Message Format



5. State and Session Management

5.1 Client-Side State Management

- **React Context API** – Global state (authentication, theme, notifications)
- **Local Storage** – API key persistence (encryption optional)
- **Session Storage** – Temporary scan form data

5.2 Server-Side Session Management

- **Stateless API** – No server-side sessions
- **Authentication** – API key passed in X-API-Key header (validated per request)
- **API Key Hashing** – bcrypt (cost factor 12)

5.3 WebSocket Connection Management

- Connection established per scan ID
- Heartbeat every 30 seconds (ping/pong)
- Automatic reconnection with exponential backoff
- Connections closed after scan completion or 1 hour idle

5.4 Task State Management

- Celery task state stored in Redis backend
- States: PENDING → STARTED → SUCCESS / FAILURE
- Task ID maps to scan ID for lookup
- Failed tasks retry up to 3 times with exponential backoff (60s, 300s, 900s)

7. Caching

6.1 Redis Cache Architecture

Parameter	Value

Host	`tejas-redis:6379`
Max Memory	512 MB
Eviction Policy	`allkeys-lru`
Persistence	AOF (Append-Only File) – every second

6.2 Cache Keys and TTL

Key Pattern	TTL	Description
`scan:{scan_id}`	300 s	Scan metadata
`scan_results:{scan_id}`	300 s	List of findings
`trends:{period}`	3600 s	Trend analysis results (1 hour)
`cve:{cve_id}`	86400 s	CVE details (24 hours)
`false_positive:{pattern_hash}`	604800 s	False positive patterns (7 days)

6.3 Cache Invalidation Strategy

- Write-through: On scan completion, cache is updated.
- Time-based: TTL expiration for all keys.
- Manual: Admin can purge cache via DELETE /api/v1/cache/{key}.

6.4 Cache Warming

- On startup, preload false positive patterns into cache.
- Scheduled task (daily) refreshes CVE cache for recent vulnerabilities.

7. Non-Functional Requirements

7.1 Security Aspects

Control	Implementation	Control
API Authentication	API key in `X-API-Key` header; bcrypt hashed storage	API Authentication
Rate Limiting	100 requests/minute per IP (NGINX)	Rate Limiting
CORS	Restricted to allowed origins (configurable)	CORS
TLS	HTTPS only (Let's Encrypt in production)	TLS
Security Headers	HSTS, CSP, X-Frame-Options, X-Content-Type-Options	Security Headers
Input Validation	Pydantic models for all API inputs	Input Validation
SQL Injection	SQLAlchemy ORM (parameterised queries)	SQL Injection
XSS Prevention	Response escaping, CSP	XSS Prevention

7.2 Performance Aspects

Metric	Target	Implementation
Scan speed	70% faster than baseline	30 concurrent requests, optimised delays
False positive rate	70-85% reduction	Multi-stage verification, confidence scoring
API response time (p95)	<200 ms	Caching, async endpoints
WebSocket latency	<100 ms	Redis Pub/Sub, dedicated workers
Concurrent scans	10 simultaneous	Celery worker pool (4 processes × multiple nodes)

Conclusion

8.1 Summary

The High-Level Design for **Tejas Raksha** presents a complete, production-ready architecture for an enterprise-grade web application security scanner. Key achievements:

- **Microservices architecture** with clear separation of concerns (frontend, API, workers, cache, database).
- **Real-time capabilities** via WebSocket and Redis Pub/Sub for live scan progress.
- **Intelligent false positive reduction** achieving 70-85% reduction through baseline detection, content signatures, and confidence scoring.
- **Extensibility** through modular security checks – new vulnerability types can be added without modifying the core engine.
- **Enterprise features** including scheduled scans, PDF reports, Slack/Jira integrations, and CVE enrichment.

8.2 Future Enhancements

- **Role-Based Access Control (RBAC)** – Multi-tenant support with admin/analyst/viewer roles.
- **JavaScript Rendering** – Integrate Playwright to test client-side vulnerabilities in SPAs.
- **API Security Testing** – Dedicated checks for REST/GraphQL APIs (rate limiting, auth bypass, injection).
- **CI/CD Integration** – GitHub Actions, GitLab CI, Jenkins plugins for automated scanning in pipelines.
- **Machine Learning** – Anomaly detection for zero-day vulnerability identification.